

DEPLOYMENT & REPRODUCTION GUIDE

Open-Source 5G NR NTN Testbed

Step-by-Step Build: 5G Core, gNB, and nrUE over an Emulated GEO
Satellite Link

Version 1.0 · August 2026

Sabbir Ahmed

Doctoral Researcher, 6G Flagship

University of Oulu, Finland

sabbiraw.ahmed@gmail.com

Companion to github.com/sabbir-uoulu/open-source-5g-ntn (DOI 10.5281/zenodo.20588732). Released under CC-BY-4.0. Reproduces the end-to-end Release-17 GEO NTN attach described in the project's reference architecture document.

Contents

1 Before you start

1.1 Host requirements · 1.2 What you will build · 1.3 Reference parameters

2 Host preparation

2.1 Base packages · 2.2 Docker · 2.3 Real-time scheduling and CPU governor

3 Part A: the 5G core (Open5GS)

3.1 Clone · 3.2 Configure · 3.3 Deploy · 3.4 Provision the subscriber · 3.5 Verify

4 Part B: the gNB (OpenAirInterface)

4.1 Clone and pin · 4.2 Apply the three patches · 4.3 Build · 4.4 Configure

5 Part C: the nrUE

5.1 UE configuration · 5.2 Common pitfalls

6 Part D: running the testbed

6.1 Start order · 6.2 gNB · 6.3 nrUE · 6.4 Data-plane test

7 Validation checklist

8 Troubleshooting

9 Teardown and known limitations

1 Before you start

1.1 Host requirements

The entire testbed runs on a single machine. No SDR or radio hardware is required; the satellite link is emulated in software.

Item	Requirement	Reference build
OS	Ubuntu 22.04 LTS (x86_64)	Ubuntu 22.04
CPU	8 cores minimum; 16+ strongly recommended	20 cores
RAM	16 GB minimum	31 GB
Disk	~40 GB free (OAI build tree is large)	—
Privileges	sudo required (real-time scheduling, Docker, tun interfaces)	—
Network	Internet access for clones, image pulls, and the data-plane test	—

IMPORTANT

Real-time thread scheduling is not optional. The RF simulator couples the gNB and UE through a lock-step sample exchange; without real-time priority the two sample clocks diverge and the random-access procedure never completes. Section 2.3 configures this, and Section 6 runs both modems under `chrt`. Skipping it is the single most common reason a rebuild of this testbed fails.

1.2 What you will build

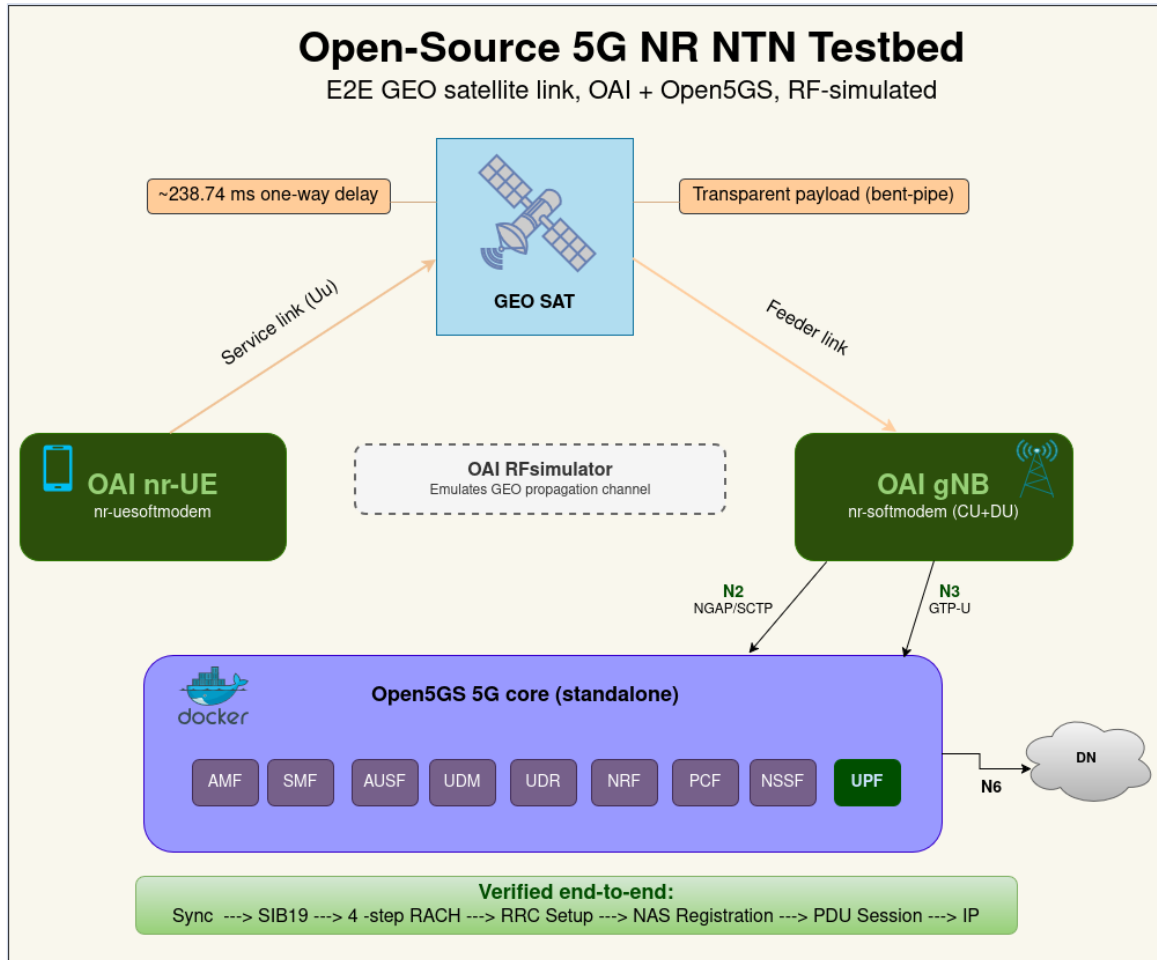


Figure 1. Target architecture. Only the UE-to-gNB link traverses the emulated satellite.

The finished system completes the full attach chain: cell synchronisation, SIB19 acquisition, four-step random access, RRC setup, NAS registration, PDU session establishment, and IP connectivity, over an emulated 238.74 ms one-way (approximately 477 ms round-trip) GEO link.

1.3 Reference parameters

These values are used consistently throughout the guide. If you change one (for example the Docker subnet), change it everywhere it appears.

Parameter	Value	Used by
PLMN (MCC/MNC)	001 / 01	Core and gNB
TAC	7	Core and gNB (must match)
S-NSSAI	SST = 1 (no SD)	Core, gNB, UE
DNN / APN	internet	Core and UE
AMF NGAP address	172.22.0.10 : 38412 (SCTP)	gNB N2 target
Host bridge to core network	172.22.0.1	gNB N2/N3 source address
UE IP pool	192.168.100.0/24	Assigned by SMF/UPF
IMSI	001010000000001	Subscriber and UE
Key (K)	fec86ba6eb707ed08905757b1bb44b8f	Subscriber and UE
OPc	c42449363bbad02b66d16bc975d77cc1	Subscriber and UE
NR band / DL frequency	Band 254 (L-band NTN) / 2 488 400 000 Hz	gNB and UE
UL offset	-873 500 000 Hz	UE
Bandwidth / SCS	25 PRB (5 MHz) / 15 kHz (numerology 0)	gNB and UE
GEO one-way delay	238.74 ms	RF simulator (both ends)
Cell-specific K-offset	478 slots	gNB SIB19
ta-Common	58 629 666 (= 238.74 ms)	gNB SIB19
OAI version	tag 2026.w16 , commit 38dc378224	gNB and UE

NOTE ON CREDENTIALS

The IMSI, K and OPc above are the standard public test vectors used with the 001/01 test PLMN. They are not secrets and are safe to commit and publish. Do not reuse them on any network carrying real traffic.

2 Host preparation

2.1 Base packages

```
sudo apt update
sudo apt install -y git build-essential cmake ninja-build pkg-config \
    libsctp-dev libssl-dev libconfig-dev iproute2 iputils-ping \
    net-tools ca-certificates curl gnupg lsb-release
```

If your shell auto-activates a Conda environment, deactivate it before any OAI build. Conda's compiler and library paths interfere with the OAI build system.

```
conda deactivate # run this in every shell used for building OAI
```

2.2 Docker

```
sudo install -m 0755 -d /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | \
  sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg
sudo chmod a+r /etc/apt/keyrings/docker.gpg
echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] \
  https://download.docker.com/linux/ubuntu $(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
  sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin docker-compose-plugin
sudo usermod -aG docker $USER      # log out and back in for this to take effect
docker --version && docker compose version
```

2.3 Real-time scheduling and CPU governor

Two host settings are required for the RF simulator's lock-step exchange to remain stable under the GEO delay.

STEP 1: RAISE THE REAL-TIME PRIORITY LIMIT (PERSISTENT)

```
sudo tee -a /etc/security/limits.conf > /dev/null <<'EOF'

# OAI RFsimulator: allow real-time thread priority
*      -      rtprio      99
*      -      memlock     unlimited
EOF
```

Log out and back in, then confirm the limit is applied:

```
ulimit -r          # expect 99 (was 0 before the change)
ulimit -Hr         # hard limit
```

STEP 2: SET THE CPU GOVERNOR TO PERFORMANCE (PER BOOT)

```
sudo bash -c 'for c in /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor; do echo performance >
$c; done'
cat /sys/devices/system/cpu/cpu0/cpufreq/scaling_governor    # expect: performance
```

NOTE

The governor setting does not survive a reboot. Re-apply it before each test session, or install it as a systemd unit. Even with `rtprio` raised, the run commands in Section 6 still launch both modems through `sudo chrt -f 80`, which sets `SCHED_FIFO` priority 80 at the process level and works regardless of the user limit.

STEP 3: REDUCE BACKGROUND LOAD

```
uptime          # check load average before running
ps -eo pid,pcpu,comm --sort=-pcpu | head -8    # identify heavy processes
```

3 Part A: the 5G core (Open5GS)

The core runs as a set of Docker containers using the `docker_open5gs` orchestration project.

3.1 Clone the deployment project

```
mkdir -p ~/NTN-deployment/external
cd ~/NTN-deployment/external
git clone https://github.com/herlesupreeth/docker_open5gs.git
cd docker_open5gs
git checkout 7722ae2eba474f50e0255fde731d0785689e37a2
```

Pinning to this commit matters: later revisions change service names, the compose file layout, and default addressing, all of which the rest of this guide assumes.

3.2 Configure the core

Configuration lives in the project's `.env` file. Create it from the supplied sample, then set the values that must match the RAN.

```
cp .env.example .env 2>/dev/null || ls -la .env      # the repo ships a sample .env
nano .env
```

Set at least the following. Leave the remaining defaults untouched unless you have a reason to change them.

Variable	Value	Why
MCC	001	Test PLMN; must match the gNB
MNC	01	Test PLMN; must match the gNB
TAC	7	Must match <code>tracking_area_code</code> in the gNB config
DOCKER_HOST_IP	your host's LAN address	Used by the WebUI and inter-service addressing
UE_IPV4_subnet	192.168.100.0/24	The pool the UE draws its address from

Confirm the Docker network addressing the RAN will target. After deployment (Section 3.3) verify with:

```
docker network inspect docker_open5gs_default | grep -E 'Subnet|Gateway'
docker inspect amf --format '{{range .NetworkSettings.Networks}}{{.IPAddress}}{{end}}'
```

ADAPT TO YOUR HOST

This guide assumes the core network is `172.22.0.0/24`, with the AMF at `172.22.0.10` and the host-side bridge gateway at `172.22.0.1`. If your Docker network differs, use your actual values in the gNB configuration (Section 4.4). The two commands above tell you what they are.

3.3 Deploy the core

```
cd ~/NTN-deployment/external/docker_open5gs
docker compose -f sa-deploy.yaml up -d
```

The first run pulls images and takes several minutes. Then confirm the network functions are up:

```
docker compose -f sa-deploy.yaml ps
docker compose -f sa-deploy.yaml ps | grep -E "amf|smf|upf" | awk '{print $1, $NF}'
```

EXPECTED

`amf` exposes `38412/sctp`; `smf` and `upf` report healthy. All containers should be in state *Up*. If any restart in a loop, inspect it with `docker compose -f sa-deploy.yaml logs <service>`.

3.4 Provision the subscriber

Open the Open5GS WebUI in a browser:

```
http://<DOCKER_HOST_IP>:9999 # default login: admin / 1423
```

Add a subscriber with exactly these values:

Field	Value
IMSI	001010000000001
Subscriber Key (K)	fec86ba6eb707ed08905757b1bb44b8f
USIM Type	OPc (not OP)
Operator Key	c42449363bbad02b66d16bc975d77cc1
DNN/APN	internet
S-NSSAI SST	1 (leave SD empty)
PDU type	IPv4 (or IPv4v6)

THE MOST COMMON SILENT FAILURE

Selecting USIM type **OP** while entering an **OPc** value produces a valid-looking subscriber that fails authentication much later, during NAS registration, with a misleading reject cause. Set the type to OPc. Section 3.5 verifies this directly in the database.

3.5 Verify the subscriber in the database

Do not rely on the WebUI alone; check the authoritative record in MongoDB.

```
docker exec mongo mongosh open5gs --quiet --eval \
'db.subscribers.find({ imsi: "001010000000001" }, \
{ imsi:1, "security.k":1, "security.opc":1, "security.op":1, \
"slice.sst":1, "slice.session.name":1, _id:0 }).pretty()'
```

EXPECTED

k matches the value above; opc is populated; **op is null**; sst is 1; session name is internet. The combination of a populated opc with a null op is the proof that the USIM type is correct.

Part A is complete: a working 5G SA core with the radio-facing NGAP listener up and one provisioned subscriber.

4 Part B: the gNB (OpenAirInterface)

4.1 Clone and pin OAI

```
cd ~/NTN-deployment/external
git clone https://gitlab.eurecom.fr/oai/openairinterface5g.git
cd openairinterface5g
git checkout 2026.w16
git log -1 --format='%h %d' # expect 38dc378224, tag 2026.w16
```

USE THIS TAG

Tag 2026.w16 is the validated baseline. Newer branches were tested during this work and introduced a UE synchronisation regression while leaving the RF simulator path unchanged; they are not a drop-in substitute.

4.2 Apply the three patches

Copy the three patch files from the project repository into a local directory, then apply them from the OAI tree root. They must all be applied before building.

```
cd ~/NTN-deployment/external/openairinterface5g

git apply ~/NTN-deployment/patches/oai-2026.w16/01-rfsim-antenna-assertion.patch
git apply ~/NTN-deployment/patches/oai-2026.w16/02-rfsim-write-queue-size.patch
git apply ~/NTN-deployment/patches/oai-2026.w16/03-ntn-rar-window-koffset.patch

# verify each landed
git status --short
grep -n "ntn_koffset" openair2/LAYER2/NR_MAC_gNB/gNB_scheduler_RA.c
```

Use `git apply --check <patch>` first if you want a dry run. What each patch does:

Patch	File	Purpose
01	radio/rfsimulator/simulator.cpp	Tolerate a zero-antenna sample header, which can arrive as a transient under the long delay; stock code aborts on it.
02	radio/COMMON/common_lib.h	Raise <code>WRITE_QUEUE_SZ</code> from 20 to 1000. The GEO timing advance queues far more pending write buffers than the stock limit allows.
03	openair2/LAYER2/NR_MAC_gNB/gNB_scheduler_RA.c	Make the Msg2/RAR response window NTN-aware by adding the cell-specific K-offset. Without this the window (max 80 ms) expires long before a 477 ms round trip completes, and random access never closes. Terrestrial behaviour is unchanged.

4.3 Build

```
conda deactivate # if Conda is active
cd ~/NTN-deployment/external/openairinterface5g/cmake_targets

./build_oai -I # first time only: install dependencies
sudo apt install -y libforms-dev libforms-bin # needed for the nrscope library

./build_oai -w SIMU --ninja --nrUE --gNB --build-lib nrscope -C
```

The full build takes considerable time. On success both binaries exist:

```
ls -la ran_build/build/nr-softmodem ran_build/build/nr-uesoftmodem
```

For later incremental rebuilds after editing a single file, build only the affected targets:

```
cd ~/NTN-deployment/external/openairinterface5g/cmake_targets/ran_build/build
ninja nr-softmodem # after a gNB MAC change (e.g. patch 03)
ninja rfsimulator nr-uesoftmodem nr-softmodem # after an RF simulator change (patches 01, 02)
```

4.4 Configure the gNB

The NTN GEO configuration ships with this OAI tag at `ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn.conf`. Edit it so the gNB reaches your core.

EDITS REQUIRED TO MATCH THE CORE

Setting	Value
tracking_area_code	7
plmn_list MCC / MNC	1 / 1 (MNC length 2)
snssaiList	a single entry { sst = 1; } with no sd
amf_ip_address	172.22.0.10
GNB_IPV4_ADDRESS_FOR_NG_AMF	172.22.0.1/24
GNB_IPV4_ADDRESS_FOR_NGU	172.22.0.1/24

NTN SETTINGS (ALREADY PRESENT IN THIS CONF; VERIFY THEM)

```
gNBs.[0]:
    min_rxtxtime           = 6;
    disable_harq           = 1;      # Rel-17: disables HARQ feedback

    servingCellConfigCommon:
        ntn-ULSyncValidityDuration-r17 = 240;
        cellSpecificOffset_r17         = 478;
        ta-Common-r17                  = 58629666;    # 238.74 ms
        positionX-r17                  = 0;
        positionY-r17                  = 0;
        positionZ-r17                  = 32433846;    # GEO radius 42 164 km
        velocityVX-r17                 = 0;
        velocityVY-r17                 = 0;
        velocityVZ-r17                 = 0;

    TIMERS:
        sr_ProhibitTimer_v1700 = 512;
        t300 = 2000; t301 = 2000; t310 = 2000; t311 = 3000; t319 = 2000;
```

RF SIMULATOR SECTION: SET THE PROPAGATION DELAY IN THE FILE, NOT ON THE COMMAND LINE

```
rfsimulator = (
{
    serveraddr = "server";
    serverport = 4043;
    options    = ();
    prop_delay = 238.74;
    IQfile     = "/tmp/rfsimulator.iqs";
}
);
```

WHY IN THE FILE

The command-line form `--rfsimulator.[0].prop_delay 238.74` is parsed inconsistently when the modem is launched through the `chrt ... env ...` wrapper used in Section 6, and an unparsed option makes the modem exit immediately with "unknown option". Setting `prop_delay` in the configuration file is reliable. Do this on both the gNB and the UE.

5 Part C: the nrUE

The UE uses the same binaries built in Section 4.3; only its configuration file differs.

5.1 UE configuration

Write `targets/PROJECTS/GENERIC-NR-5GC/CONF/ue.conf` exactly as follows.

```
# SPDX-License-Identifier: LicenseRef-CSSL-1.0
uicc0 = {
imsi = "001010000000001";
key = "fec86ba6eb707ed08905757b1bb44b8f";
opc = "c42449363bbad02b66d16bc975d77cc1";
pdu_sessions = ({ dnn = "internet"; nssai_sst = 1; });
}
position0 = {
    x = 0.0;
    y = 0.0;
    z = 6377900.0;
}
rf Simulator = (
{
    serveraddr = "127.0.0.1";
    serverport = 4043;
    options = ();
    prop_delay = 238.74;
}
);
```

The `position0` block gives the UE its ground position. A GNSS-equipped NTN UE uses this together with the ephemeris from SIB19 to compute its own timing advance, so it must be present.

5.2 Common pitfalls in the UE configuration

- **Do not include a LEO channel model in a GEO scenario.** If the file contains `@include "channelmod_rf Simu_LEO_satellite.conf"`, remove it. That model applies time-varying delay and Doppler for a 600 km orbit and is wrong here; for transparent GEO the constant `prop_delay` is all that is needed.
- **Credentials must match the provisioned subscriber exactly**, including the DNN and SST.
- **Set `prop_delay` here as well as on the gNB.** Both ends apply the delay.

6 Part D: running the testbed

6.1 Start order

Start in this order, in three terminals. Each component must be up before the next is started.

1. Core network (Section 3.3), verified with `docker compose -f sa-deploy.yaml ps`
2. gNB, which acts as the RF simulator server
3. nrUE, which connects to it as the client

Before each session, re-apply the CPU governor if the host has rebooted:

```
sudo bash -c 'for c in /sys/devices/system/cpu/cpu*/cpufreq/scaling_governor; do echo performance > $c; done'
```

6.2 Terminal 2: the gNB

```
cd ~/NTN-deployment/external/openairinterface5g/cmake_targets

sudo chrt -f 80 env RFSIMULATOR=server ./ran_build/build/nr-softmodem \
-O ../ci-scripts/conf_files/gnb.sa.band254.u0.25prb.rfsim.ntn.conf \
--rfsim --gNBs.[0].min_rtxtime 6 2>&1 | tee /tmp/gnb.log
```

WAIT FOR THESE LINES BEFORE STARTING THE UE

```
Received NGSetupResponse from AMF — the N2 link to the core is up
cell PLMN 001.01 Cell ID ... is in service
propagation delay 238.740104 ms, 1833524 samples — the GEO delay loaded from the conf
Running as server waiting opposite rfsimulators to connect
```

6.3 Terminal 3: the nrUE

```
cd ~/NTN-deployment/external/openairinterface5g/cmake_targets

sudo chrt -f 80 env RFSIMULATOR=127.0.0.1 ./ran_build/build/nr-uesoftmodem \
  -O ../targets/PROJECTS/GENERIC-NR-5GC/CONF/ue.conf \
  --band 254 -C 2488400000 --CO -873500000 \
  -r 25 --numerology 0 --ssb 60 --rfsim 2>&1 | tee /tmp/ue.log
```

Allow up to 60 seconds. Random access over a GEO round trip is slow by nature, and the procedure may take several attempts before it completes.

6.4 Data-plane test

Once the UE reports its tunnel interface, test IP connectivity from a fourth terminal.

```
ip addr show oaitun_ue1                # expect an address in 192.168.100.0/24
ping -I oaitun_ue1 -c 10 192.168.100.1 # the UPF gateway
ping -I oaitun_ue1 -c 10 8.8.8.8        # out through N6
```

ON THE MEASURED ROUND-TRIP TIME

The reported ping time will be tens of milliseconds, not the approximately 477 ms of a physical GEO link. The propagation delay is modelled faithfully in the sample and timing domain, which is what drives all the NTN protocol behaviour, but the RF simulator is not a real-time system: it processes samples as fast as the host allows. The delay is real where it determines protocol correctness; it is not a wall-clock latency emulator. This is a property of the RF simulator, not a fault in the deployment.

7 Validation checklist

Work through these in order. Each stage depends on the previous one, so the first failure tells you where to look.

#	Stage	Command / where	Expected evidence
1	Core up	<code>docker compose -f sa-deploy.yaml ps</code>	amf/smf/upf Up; amf on 38412/sctp
2	Subscriber correct	mongosh query, Section 3.5	<code>opc set, op null, sst 1, dnn internet</code>
3	N2 link	gNB log	Received NGSetupResponse from AMF
4	Cell in service	gNB log	<code>cell PLMN 001.01 ... is in service; activate SIB19 at DU</code>
5	GEO delay loaded	gNB and UE logs	propagation delay 238.740104 ms, 1833524 samples
6	UE synchronised	UE log	Initial sync successful; PBCH decoded
7	SIB19 and NTN params	UE log	k_offset 478; N_Common_TA 238.74 ms; timing_advance_ntn 3667039 samples
8	Random access	UE log	Found RAR with the intended RAPID; RA-Msg3 transmitted; 4-Step RA procedure succeeded. CBRA: Contention Resolution is successful.
9	RRC setup	UE log	Received NR_RRCSetup; Generating RRCSetupComplete
10	NAS registration	UE log	Received Registration Accept with result 3GPP
11	PDU session	gNB log	Added PDU Session 1; Added DRB 1; NGAP_PDUSESSION_SETUP_RESP
12	IP connectivity	UE log, then ping	TUN Interface oaitun_ue1 successfully configured; ping 0% loss

Quick extraction of the decisive lines after a run:

```
grep -iE "Found RAR with the intended RAPID|RA-Msg3 transmitted|RA procedure succeeded|\
Contention Resolution|Received NR_RRCSetup|RRCSetupComplete|Registration Accept|\
TUN Interface" /tmp/ue.log

grep -iE "NGSetupResponse|is in service|activate SIB19|received PDU Session Resource Setup Request|\
Added PDU Session|Added DRB|NGAP_PDUSESSION_SETUP_RESP|RRCReconfigurationComplete" /tmp/gnb.log
```

8 Troubleshooting

The UE exits immediately with "unknown option"

The RF simulator options were passed on the command line through the `chrt ... env` wrapper and were not parsed. Remove `--rfsimulator.[0].prop_delay` from the command line and set `prop_delay` in both configuration files instead (Sections 4.4 and 5.1).

The gNB never reports receiving a preamble; the UE loops on "RAR reception failed"

Almost always real-time scheduling. Confirm all three: `ulimit -r` is not 0; the CPU governor is `performance`; and both modems were started with `sudo chrt -f 80`. Without real-time priority the two sample clocks diverge and the uplink never lands in the gNB's read window. Also check that patch 03 was applied and compiled in (`grep -n ntn_koffset openair2/LAYER2/NR_MAC_gNB/gNB_scheduler_RA.c`, then rebuild).

"Received RAR preamble (X) doesn't match the intended RAPID (Y)"

This is the signature of patch 03 missing or not rebuilt. The RAR window expires before the round trip completes, the UE retries with a new preamble, and the late RAR is discarded as a mismatch. Apply patch 03 and run `ninja nr-softmodem`.

NAS registration is rejected

Check the subscriber record (Section 3.5), specifically that the USIM type is OPc and `op` is null. Then confirm the UE's IMSI, key, OPc, DNN and SST match the record exactly.

The gNB cannot reach the AMF

Verify the AMF's container address and your Docker subnet with the commands in Section 3.2, then update `amf_ip_address` and the two `GNB_IPV4_ADDRESS_FOR_*` values. Confirm the SCTP port is listening: `ss -lnp | grep 38412`.

The gNB starts but the cell is not in service

Usually a PLMN, TAC, or slice mismatch between the gNB configuration and the core. All three must be identical on both sides.

The build fails in an unexpected way

Confirm Conda is deactivated in the build shell, that you are on tag `2026.w16`, and that `./build_oai` is completed. A clean rebuild is `./build_oai -w SIMU --ninja --nrUE --gNB --build-lib nrscope -C`.

The session establishes, then the gNB sends an RRC Release

An inactivity release with no traffic on the bearer. Start the ping promptly after the tunnel interface appears, or extend the inactivity timer.

9 Teardown and known limitations

9.1 Teardown

```
# stop the modems with Ctrl+C in their terminals, then:
cd ~/NTN-deployment/external/docker_open5gs
docker compose -f sa-deploy.yaml down          # add -v to also drop the subscriber database
```

9.2 Known limitations

- **RF simulation only.** No radio is transmitted; the satellite channel is a software model of propagation delay.
- **Wall-clock latency does not reflect GEO delay**, for the reason given in Section 6.4. The delay is faithful in the sample and timing domain, which is what the NTN protocol features exist to handle, but the simulator is not real-time.
- **Transparent GEO payload only.** LEO introduces time-varying delay and significant Doppler and exercises a different part of the Release-17 toolkit; it is out of scope here.
- **Single host, no scale or performance characterisation**, and no certification or regulatory work.
- **Patch scope.** All three patches were applied together to produce the working build. They have not been individually isolation-tested to establish whether patches 01 and 02 are each strictly required or partly defensive.

9.3 Further reading

The project repository at github.com/sabbir-uoulu/open-source-5g-ntn contains the tracked configurations, the three patch files with full rationale headers, pinned dependency versions, complete laboratory notes covering how each problem was diagnosed, and the reference architecture

document (DOI 10.5281/zenodo.20588732) describing the system architecture and Release-17 procedures in detail.